

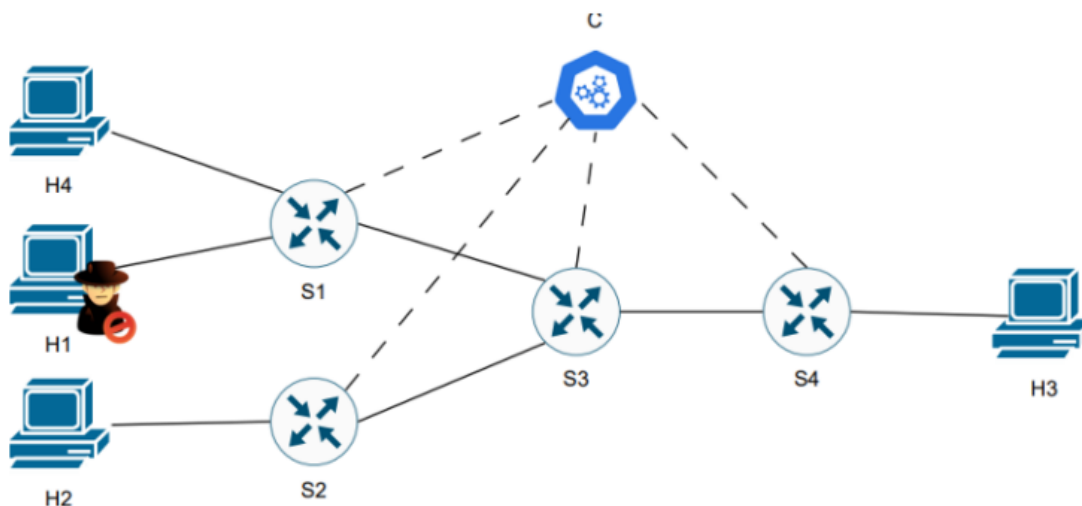
Progetto NCIS – Calio Angelo

Progettazione e Implementazione di un Firewall SDN per la Mitigazione di Attacchi DoS

Introduzione e Obiettivi del progetto

L'obiettivo di questo progetto è progettare, emulare e validare un sistema di Firewalling e Traffic Monitoring SDN in grado di intervenire in modo da garantire le funzionalità del resto degli utenti.

Topologia



L'infrastruttura è emulata mediante l'ambiente Mininet ed è costituita da:

- Host **H1**: host ostile generatore di attacco DoS, inviando un traffico UDP a banda costante
- Host **H2** e **H4**: host legittimi che generano traffico generico TCP a banda costante
- Host **H3**: target server eroga Servizi e riceve flussi

I client sono connessi fra di loro tramite 4 switch Open vSwitch, gestiti da un controller Openflow SDN.

Per definire la topologia si è utilizzato **Mininet**:

```
#!/usr/bin/python
import threading
import random
import time
from mininet.log import setLogLevel, info
from mininet.topo import Topo
from mininet.net import Mininet, CLI
from mininet.node import OVSKernelSwitch, Host
from mininet.link import TCLink, Link
from mininet.node import RemoteController #Controller
from mininet.util import dumpNodeConnections

class Environment(object):
    def __init__(self):

        # Creiamo una rete in Mininet, specificando un controller remoto e definendo i
        # Link con interfaccia TC Link
        # TCLink (Traffic Control Link), che permette di emulare parametri fisici di
        # rete
```

```

        # (come limitazione di banda, delay e packet loss) tramite lo strumento di
sistema 'tc'.
        "Create a network."
        self.net = Mininet(controller=RemoteController, link=TCLink)

        try:
            # Aggiungiamo un controller remoto alla rete
            # Il controller remoto fornito dalla libreria agisce indipendentemente da
Mininet
            info("*** Starting controllers\n")
            c1 = self.net.addController( 'C1', controller=RemoteController, port=6633)
#Controller 1: Protegge dai flussi in eccesso
            c1.start()

            # Aggiungiamo gli host
            info("*** Adding hosts\n")
            self.h1 = self.net.addHost('h1', mac = '00:00:00:00:00:01', ip= '10.0.0.1')
            self.h2 = self.net.addHost('h2', mac = '00:00:00:00:00:02', ip= '10.0.0.2')
            self.h3 = self.net.addHost('h3', mac = '00:00:00:00:00:03', ip= '10.0.0.3')
            self.h4 = self.net.addHost('h4', mac = '00:00:00:00:00:04', ip= '10.0.0.4')

            # Aggiungiamo gli switch basati su Open VSwitch
            info("*** Adding switches\n")
            self.s1 = self.net.addSwitch('s1')
            self.s2 = self.net.addSwitch('s2')
            self.s3 = self.net.addSwitch('s3')
            self.s4 = self.net.addSwitch('s4')

            # Colleghiamo il tutto, limitando la banda a 10mbps
            info("*** Adding links\n")
            self.net.addLink(self.h1,self.s1)
            self.net.addLink(self.h4,self.s1)
            self.net.addLink(self.h2,self.s2)
            self.net.addLink(self.h3,self.s4)
            self.net.addLink(self.s1,self.s3,bw=10)
            self.net.addLink(self.s2,self.s3,bw=5)
            self.net.addLink(self.s4,self.s3,bw=15)

            # Costruzione e avvio della topologia
            info("*** Starting network\n")
            self.net.build()
            self.net.start()
            self.net.pingAll()
            self.h3.cmd('iperf -s&') # Per testare il traffico TCP

        except:
            self.net.stop()

    ...
if __name__ == '__main__':

    setLogLevel('info')
    info('starting the environment\n')
    env = Environment()

    info("*** Running CLI\n")
    CLI(env.net)
    env.net.stop()

```

Controller

Il modulo di controllo estende SimpleSwitch13 di Ryu, integrando un processo di analisi del traffico, di calcolo dei throughput e di rilevazione e mitigazione del traffico in eccesso.

```

from operator import attrgetter
from ryu import cfg
from ryu.app import simple_switch_13
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER, DEAD_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.lib import hub

class SimpleMonitor13(simple_switch_13.SimpleSwitch13):

    MAX_NODES = 100
    THRESHOLD = 50*1000*1000 #50 MBits/s
    SLEEP_TIME = 10 # Periodo di attesa tra i monitoraggi
    WAIT_TIME = 5 # Numero di cicli (SLEEP_TIME) di attesa prima che si ripristini il
traffico su una porta

    CONF = cfg.CONF
    CONF.ofp_tcp_listen_port = 6633

    def __init__(self, *args, **kwargs):
        super(SimpleMonitor13, self).__init__(*args, **kwargs)
        self.datapaths = {}
        self.database = [None] * self.MAX_NODES
        self.sleep_counter = [[-1 for _ in range(self.MAX_NODES)] for _ in
range(self.MAX_NODES)]
        self.monitor_thread = hub.spawn(self._monitor)

    @set_ev_cls(ofp_event.EventOFPPortStatsRequest,
[MAIN_DISPATCHER, DEAD_DISPATCHER])
    def _port_stats_request_handler(self, ev):
        datapath = ev.datapath
        if ev.state == MAIN_DISPATCHER:
            if datapath.id not in self.datapaths:
                self.logger.debug('register datapath: %016x', datapath.id)
                self.datapaths[datapath.id] = datapath
            elif ev.state == DEAD_DISPATCHER:
                if datapath.id in self.datapaths:
                    self.logger.debug('unregister datapath: %016x', datapath.id)
                    del self.datapaths[datapath.id]

    def _monitor(self):
        while True:
            for dp in sorted(self.datapaths.values(), key=attrgetter('id')):
                self._request_stats(dp)
                hub.sleep(self.SLEEP_TIME)

    def _request_stats(self, datapath):
        self.logger.debug('send stats request: %016x', datapath.id)
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        req = parser.OFPPortStatsRequest(datapath, 0, ofproto.OFPP_ANY)
        datapath.send_msg(req)

    @set_ev_cls(ofp_event.EventOFPPortStatsReply, MAIN_DISPATCHER)
    def _port_stats_reply_handler(self, ev):
        body = ev.msg.body
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        if self.database[datapath.id] == None:
            self.database[datapath.id] = [None]*(len(body)+1)
            measures = self.database[datapath.id]

```

```

self.logger.info('||-----||')
-----||')
self.logger.info('||datapath          port '
                ' rx-pkts      rx-bytes rx-error  rx-throughput '
                ' tx-pkts      tx-bytes tx-error  tx-throughput||')
self.logger.info('||----- '
                '----- '
                '-----||')

for stat in sorted(body, key=attrgetter('port_no')):

    if stat.port_no == int('FFFFFFFE', 16):
        port = 0
    else:
        port = stat.port_no

    if measures[port] == None or measures[port].rx_packets > stat.rx_packets:
        measures[port] = stat

    # Calcolo throughput
    tx_throughput = (stat.tx_bytes-measures[port].tx_bytes)/self.SLEEP_TIME*8
    rx_throughput = (stat.rx_bytes-measures[port].rx_bytes)/self.SLEEP_TIME*8

    if tx_throughput < 1000:
        tx_throughput_str = str(round(tx_throughput,2)) + " Bits/s"
    elif tx_throughput < 1000000:
        tx_throughput_str = str(round(tx_throughput/1000, 2)) + " KBits/s"
    elif tx_throughput < 1000000000:
        tx_throughput_str = str(round(tx_throughput/1000000, 2)) + " MBits/s"
    else:
        tx_throughput_str = str(round(tx_throughput/1000000000, 2)) + " GBits/s"

    if rx_throughput < 1000:
        rx_throughput_str = str(round(rx_throughput,2)) + " Bits/s"
    elif rx_throughput < 1000000:
        rx_throughput_str = str(round(rx_throughput/1000, 2)) + " KBits/s"
    elif rx_throughput < 1000000000:
        rx_throughput_str = str(round(rx_throughput/1000000, 2)) + " MBits/s"
    else:
        rx_throughput_str = str(round(rx_throughput/1000000000, 2)) + " GBits/s"

    self.logger.info('||' + '{:016d}{:> 9X}{:> 10n}{:> 14n}{:> 10n}{:>16s}{:>
10n}{:> 14n}{:> 10n}{:>16s}'.format(
        ev.msg.datapath.id, stat.port_no,
        stat.rx_packets, stat.rx_bytes, stat.rx_errors,
rx_throughput_str,
        stat.tx_packets, stat.tx_bytes, stat.tx_errors,
tx_throughput_str) + '||')

    # Notifica su S3 in caso di attacco
    if rx_throughput > self.THRESHOLD:
        self.logger.warning('Anomaly detected! High traffic on port %d',
stat.port_no)
        # Crea una regola di match per tutti i pacchetti provenienti dalla porta
`port_no`
        match = parser.OFPMatch(in_port=stat.port_no)
        # Nessuna azione, drop dei pacchetti
        actions = []
        # Istruzione per non eseguire nessuna azione (drop)
        inst = [parser.OFPIInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
actions)]
        # Crea un messaggio OFPFlowMod per aggiungere la regola
        mod = parser.OFPFlowMod(datapath=datapath, priority=2, match=match,
instructions=inst)
        # Invia il messaggio allo switch

```

```

        datapath.send_msg(mod)
        self.sleep_counter[datapath.id][port] = self.WAIT_TIME
    elif self.sleep_counter[datapath.id][port] == 0:
        self.logger.warning('Resumed traffic on port %d', stat.port_no)
        match = parser.OFPMatch(in_port=stat.port_no)
        # Ripristina il flusso predefinito con azione forward (invio normale)
        actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)] # Azione per
inviare normalmente i pacchetti
        # Istruzione per non eseguire nessuna azione (drop)
        inst = [parser.OFPIInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
actions))]
        # Crea un messaggio OFPFlowMod per ripristinare il flusso
        mod = parser.OFPFlowMod(datapath=datapath, priority=1, match=match,
instructions=inst, command=ofproto.OFPPC_MODIFY)
        # Invia il messaggio allo switch per ripristinare il traffico
        datapath.send_msg(mod)
    else:
        self.sleep_counter[datapath.id][port] -= 1

# Salviamo e aggiorniamo le misurazioni
measures[port] = stat
self.database[datapath.id] = measures

```

Esecuzione

L'esecuzione è stata avviata su un sistema virtualizzato con in esecuzione Ubuntu 22.04 LTS con python versione 3.9.

Creazione della Topologia

Si definisce la topologia, testandola usando un controller generico SimpleSwitch13.

Definizione della Topologia

Si inizializza la topologia attraverso la configurazione Mininet e la creazione del controller Ryu.

```

starting the environment
*** Starting controllers
*** Adding hosts
*** Adding switches
*** Adding links
(10.00Mbit) (10.00Mbit) (5.00Mbit) (5.00Mbit) (15.00Mbit) (15.00Mbit) *** Starting network
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
C1
*** Starting 4 switches
s1 (10.00Mbit) s2 (5.00Mbit) s3 (10.00Mbit) (5.00Mbit) (15.00Mbit) s4 (15.00Mbit) ... (10.00Mbit) (5.00Mbit) (10.00Mbit)
(5.00Mbit) (15.00Mbit) (15.00Mbit)

```

Verifica della raggiungibilità degli host

pingAll() permette di testare la comunicazione reciproca fra tutti e 4 gli host.

```

*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>

```

Generazione del traffico

Per generare il traffico si utilizza il comando iperf -c 10.0.0.3 inviando il traffico di rete verso l'host H3 (che ha ip locale 10.0.0.3 come definito dalla topologia).

H2 e H4 inviano traffico TCP con iperf -c 10.0.0.3 direttamente: la banda è limitata automaticamente tramite la configurazione degli switch.

H1 invia traffico UDP con iperf -c 10.0.0.3 -u.

```
[ 1] 48.0000-49.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 49.0000-50.0000 sec 1.06 MBytes 8.86 Mbits/sec
[ 1] 50.0000-51.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 51.0000-52.0000 sec 1.06 MBytes 8.86 Mbits/sec
[ 1] 52.0000-53.0000 sec 1.18 MBytes 9.90 Mbits/sec
[ 1] 53.0000-54.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 54.0000-55.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 55.0000-56.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 56.0000-57.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 57.0000-58.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 58.0000-59.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 59.0000-60.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 60.0000-61.0000 sec 1.18 MBytes 9.90 Mbits/sec
[ 1] 61.0000-62.0000 sec 573 KBytes 4.69 Mbits/sec
[ 1] 62.0000-63.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 63.0000-64.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 64.0000-65.0000 sec 1.18 MBytes 9.90 Mbits/sec
[ 1] 65.0000-66.0000 sec 633 KBytes 5.19 Mbits/sec
[ 1] 66.0000-67.0000 sec 63.6 KBytes 521 Kbits/sec
[ 1] 67.0000-68.0000 sec 127 KBytes 1.04 Mbits/sec
[ 1] 68.0000-69.0000 sec 255 KBytes 2.09 Mbits/sec
[ 1] 69.0000-70.0000 sec 127 KBytes 1.04 Mbits/sec
[ 1] 70.0000-71.0000 sec 127 KBytes 1.04 Mbits/sec
[ 1] 71.0000-72.0000 sec 255 KBytes 2.09 Mbits/sec
[ 1] 72.0000-73.0000 sec 127 KBytes 1.04 Mbits/sec
[ 1] 73.0000-74.0000 sec 255 KBytes 2.09 Mbits/sec
```

Simulazione attacco DoS

Esecuzione del comando iperf -c 10.0.0.3 -u -b 50G -P 100, impostando una banda richiesta spropositata e flussi paralleli. In questo modo il link di transito si satura, si osserva infatti che gli altri client subiscono un drastico calo del throughput, una consistente perdita di pacchetti e, nei momenti di picco, una completa starvation del traffico.

```
[ 1] 10.0000-11.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 11.0000-12.0000 sec 1.18 MBytes 9.90 Mbits/sec
[ 1] 12.0000-13.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 13.0000-14.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 14.0000-15.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 15.0000-16.0000 sec 954 KBytes 7.81 Mbits/sec
[ 1] 16.0000-17.0000 sec 1.12 MBytes 9.39 Mbits/sec
[ 1] 17.0000-18.0000 sec 573 KBytes 4.69 Mbits/sec
[ 1] 18.0000-19.0000 sec 1.18 MBytes 9.90 Mbits/sec
[ 1] 19.0000-20.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 20.0000-21.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 21.0000-22.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 22.0000-23.0000 sec 1017 KBytes 8.33 Mbits/sec
[ 1] 23.0000-24.0000 sec 1.12 MBytes 9.39 Mbits/sec
[ 1] 24.0000-25.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 25.0000-26.0000 sec 1.12 MBytes 9.38 Mbits/sec
[ 1] 26.0000-27.0000 sec 697 KBytes 5.71 Mbits/sec
[ 1] 27.0000-28.0000 sec 127 KBytes 1.04 Mbits/sec
[ 1] 28.0000-29.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 29.0000-30.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 30.0000-31.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 31.0000-32.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 32.0000-33.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 33.0000-34.0000 sec 0.000 Bytes 0.000 bits/sec
```

Attivazione Controller

Adesso viene inserito il controller progettato

Monitoraggio del traffico

Il controller mostra a schermo il traffico lungo i nodi con una periodicità dettata dalla variabile SLEEP_TIME:

- datapath indica lo switch analizzato
- port indica la porta dello switch per cui passano i pacchetti
- rx-pkts e tx-pkts rappresentano i pacchetti in ingresso e uscita nel nodo lungo la specifica porta in totale
- rx-bytes e tx-bytes rappresentano il numero byte in ingresso e uscita nel nodo lungo la specifica porta in totale
- rx-error e tx-error rappresentano il numero di pacchetti che presentano errori
- rx-throughput e tx-throughput indicano il flusso effettivo di dati in ingresso e uscita nel momento

datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000001	1	28	2284	0	0.0 Bits/s	123	14602	0	56.0 Bits/s
0000000000000001	2	29	2374	0	0.0 Bits/s	122	14536	0	56.0 Bits/s
0000000000000001	3	107	11566	0	56.0 Bits/s	63	6030	0	0.0 Bits/s
0000000000000001	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000003	1	63	6030	0	0.0 Bits/s	107	11566	0	56.0 Bits/s
0000000000000003	2	49	4942	0	56.0 Bits/s	109	11658	0	0.0 Bits/s
0000000000000003	3	49	4942	0	0.0 Bits/s	108	11572	0	56.0 Bits/s
0000000000000003	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000002	1	29	2370	0	56.0 Bits/s	131	14430	0	0.0 Bits/s
0000000000000002	2	109	11658	0	0.0 Bits/s	49	4942	0	56.0 Bits/s
0000000000000002	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000004	1	28	2284	0	0.0 Bits/s	131	14430	0	56.0 Bits/s
0000000000000004	2	108	11572	0	56.0 Bits/s	49	4942	0	0.0 Bits/s
0000000000000004	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s

Simulazione d'attacco da parte di H1:

datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000004	1	633	49022	0	2.87 KBits/s	1891	39114670	0	29.03 KBits/s
0000000000000004	2	1864	39111458	0	29.03 KBits/s	658	52034	0	2.87 KBits/s
0000000000000004	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000002	1	31	2510	0	56.0 Bits/s	153	16266	0	0.0 Bits/s
0000000000000002	2	127	13140	0	0.0 Bits/s	54	5366	0	0.0 Bits/s
0000000000000002	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000003	1	1818	39117234	0	38.71 KBits/s	725	59436	0	2.87 KBits/s
0000000000000003	2	54	5366	0	0.0 Bits/s	127	13140	0	0.0 Bits/s
0000000000000003	3	658	52034	0	2.87 KBits/s	1872	39123554	0	38.71 KBits/s
0000000000000003	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000001	1	69559041	105173224116	0	82.57 GBits/s	168	24080	0	2.87 KBits/s
anomaly detected! High traffic on port 1									
0000000000000001	2	633	37856810	0	1.21 KBits/s	742	55188	0	0.0 Bits/s
0000000000000001	3	725	59436	0	2.87 KBits/s	1810	39105138	0	29.03 KBits/s
0000000000000001	FFFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s

Con un throughput massimo definito tramite la variabile threshold e un sistema di monitoraggio capace di segnalare quando si ha un traffico anomal osserviamo che H1 genera un traffico eccessivo sulla porta 1 dello switch 1.

Data la banda definita nella topologia, la threshold è definita intorno ai 50Mbits/s.

Mitigazione

Per risolvere il problema si individua lo switch e la porta corrispondenti a quelli sotto attacco: si definisce una direttiva per dropare tutti i pacchetti in ingresso nella singola porta e si imposta un “countdown”, un numero di misurazioni da effettuare di fila senza superare nuovamente la threshold prima di poter riaprire la porta chiusa.

Il traffico dei client torna allo stato di normalità per via della assenza di saturazione della rete.

[1]	68.0000-69.0000	sec	891	KBytes	7.30	Mbits/sec
[1]	69.0000-70.0000	sec	1.12	MBytes	9.38	Mbits/sec
[1]	70.0000-71.0000	sec	1.12	MBytes	9.38	Mbits/sec
[1]	71.0000-72.0000	sec	573	KBytes	4.69	Mbits/sec
[1]	72.0000-73.0000	sec	1.06	MBytes	8.87	Mbits/sec
[1]	73.0000-74.0000	sec	124	KBytes	1.01	Mbits/sec
[1]	74.0000-75.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	75.0000-76.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	76.0000-77.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	77.0000-78.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	78.0000-79.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	79.0000-80.0000	sec	0.000	Bytes	0.000	bits/sec
[1]	80.0000-81.0000	sec	128	KBytes	1.05	Mbits/sec
[1]	81.0000-82.0000	sec	575	KBytes	4.71	Mbits/sec
[1]	82.0000-83.0000	sec	954	KBytes	7.81	Mbits/sec
[1]	83.0000-84.0000	sec	954	KBytes	7.82	Mbits/sec
[1]	84.0000-85.0000	sec	1.06	MBytes	8.87	Mbits/sec
[1]	85.0000-86.0000	sec	954	KBytes	7.81	Mbits/sec
[1]	86.0000-87.0000	sec	1.06	MBytes	8.87	Mbits/sec

Infine, verrà inviata una notifica dal controller con annessa riapertura della porta, appena h1 torna ad avere un comportamento ammissibile nella rete per un quantitativo di tempo sufficiente permettendo la riconnessione.

datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000003	1	2958	43531567	0	0.0 Bits/s	914	79693	0	0.0 Bits/s
0000000000000003	2	57	5613	0	0.0 Bits/s	139	13979	0	0.0 Bits/s
0000000000000003	3	842	71867	0	0.0 Bits/s	3017	43538311	0	0.0 Bits/s
0000000000000003	FFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000002	1	31	2510	0	0.0 Bits/s	167	17282	0	0.0 Bits/s
0000000000000002	2	139	13979	0	0.0 Bits/s	57	5613	0	0.0 Bits/s
0000000000000002	FFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000004	1	815	68678	0	0.0 Bits/s	3046	43541700	0	0.0 Bits/s
0000000000000004	2	3017	43538311	0	0.0 Bits/s	842	71867	0	0.0 Bits/s
0000000000000004	FFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s
datapath	port	rx-pkts	rx-bytes	rx-error	rx-throughput	tx-pkts	tx-bytes	tx-error	tx-throughput
0000000000000001	1	111507096	168598643614	0	0.0 Bits/s	198	32920	0	0.0 Bits/s
Resumed traffic on port 1									
0000000000000001	2	799	40802746	0	0.0 Bits/s	918	67868	0	0.0 Bits/s
0000000000000001	3	914	79693	0	0.0 Bits/s	2958	43531567	0	0.0 Bits/s
0000000000000001	FFFFFFFE	0	0	0	0.0 Bits/s	0	0	0	0.0 Bits/s